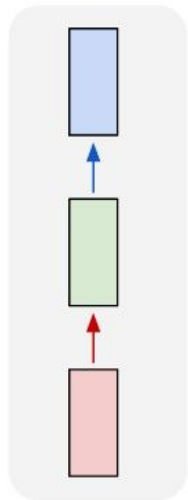


Обработка естественного языка: RNN

“Vanilla” Neural Network

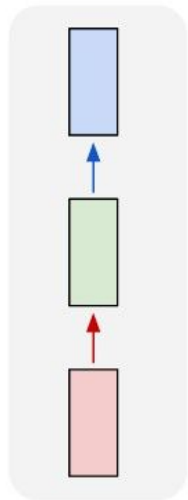
one to one



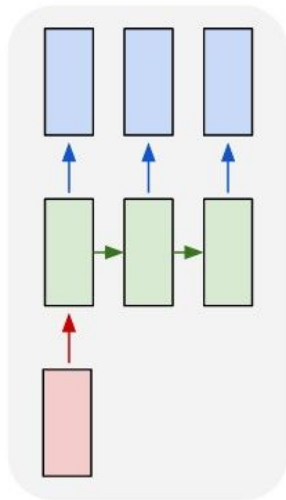
Vanilla Neural Networks

Recurrent Neural Networks: Process Sequences

one to one



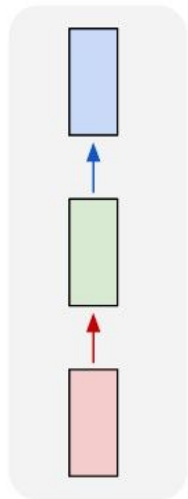
one to many



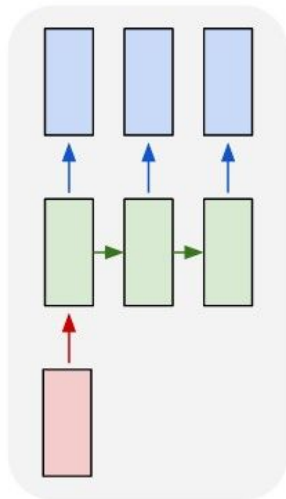
↖ e.g. **Image Captioning**
image -> sequence of words

Recurrent Neural Networks: Process Sequences

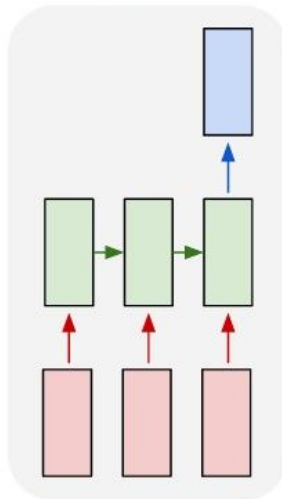
one to one



one to many



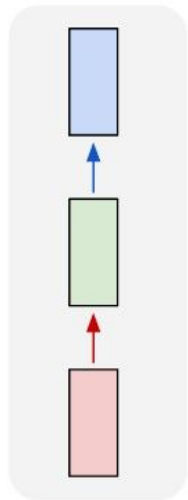
many to one



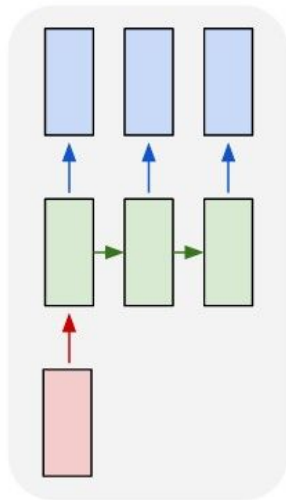
e.g. **action prediction**
sequence of video frames -> action class

Recurrent Neural Networks: Process Sequences

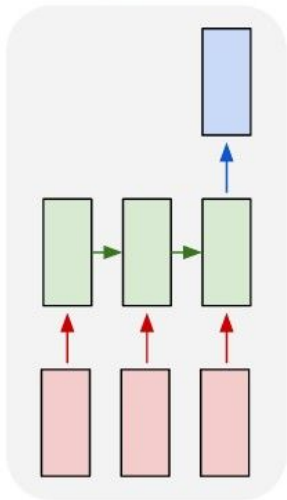
one to one



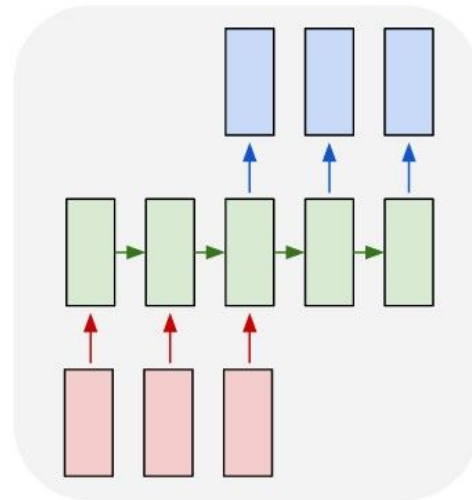
one to many



many to one



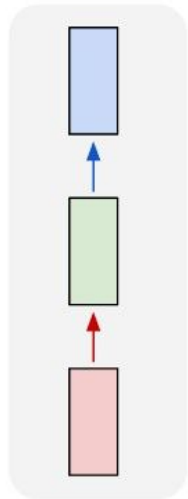
many to many



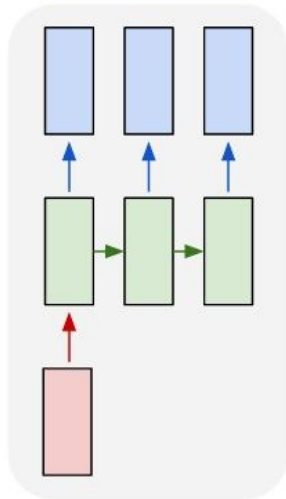
E.g. **Video Captioning**
Sequence of video frames ->
caption

Recurrent Neural Networks: Process Sequences

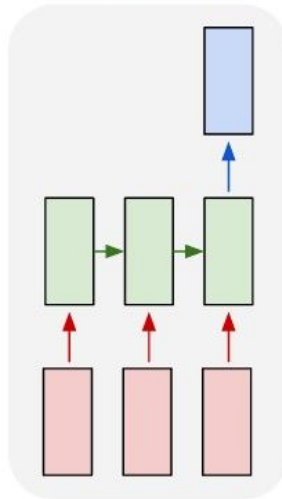
one to one



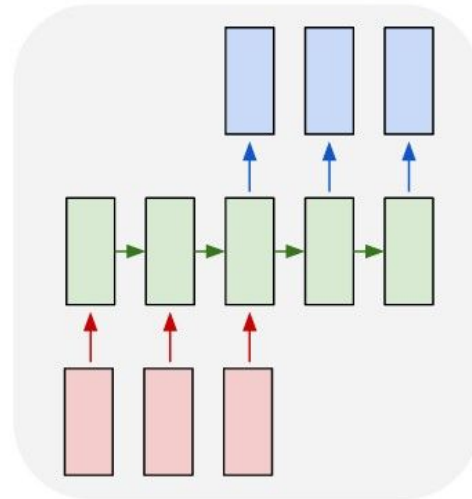
one to many



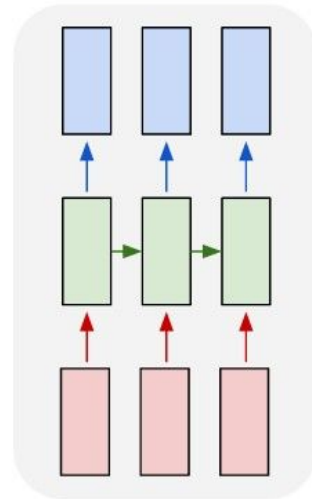
many to one



many to many



many to many



e.g. Video classification on frame level



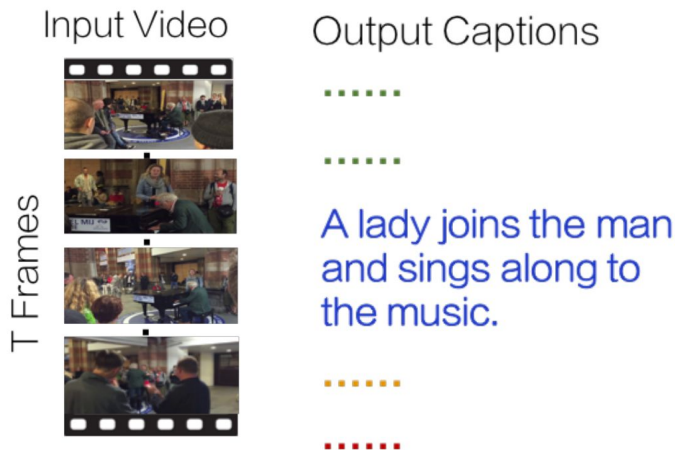
Why existing convnets are insufficient?

Variable sequence length inputs and outputs!

Example task: video captioning

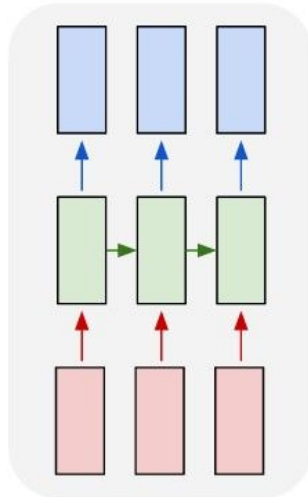
Input video can have variable number of frames

Output captions can be variable length.

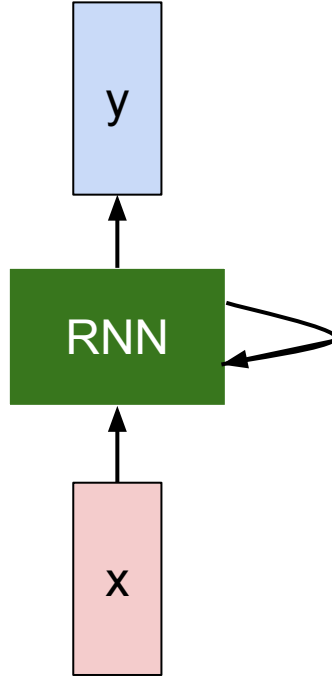


Let's start with a task that takes a variable input and produces an output at every step

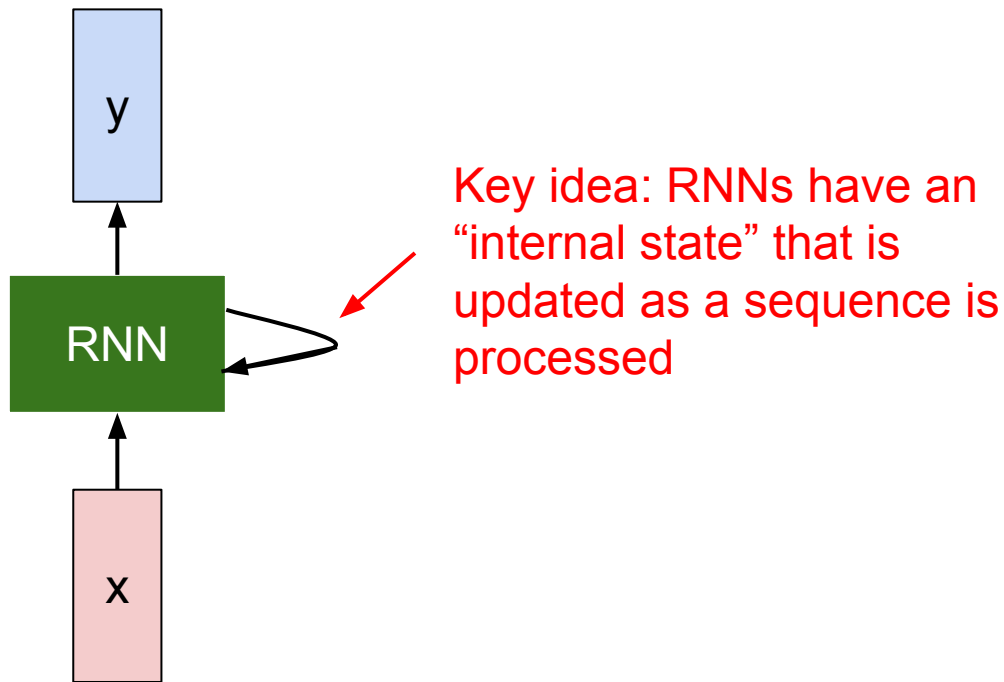
many to many



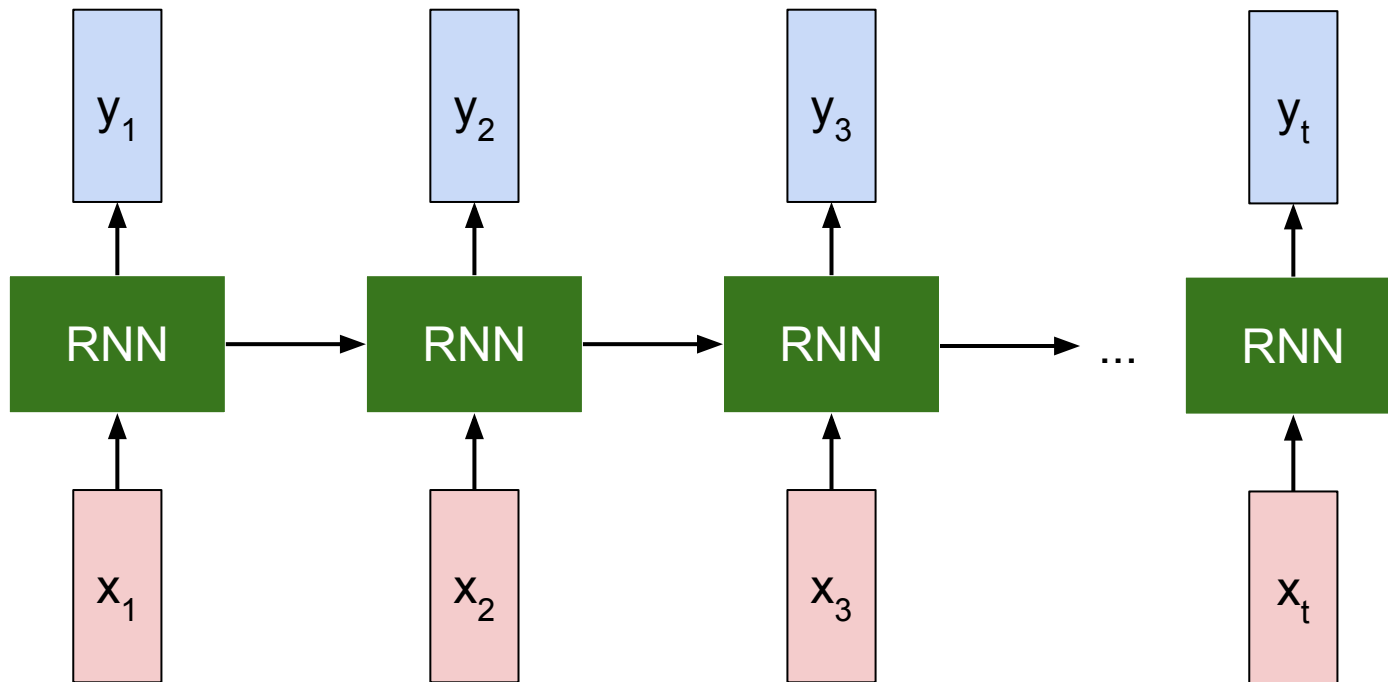
Recurrent Neural Network



Recurrent Neural Network



Unrolled RNN



RNN hidden state update

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

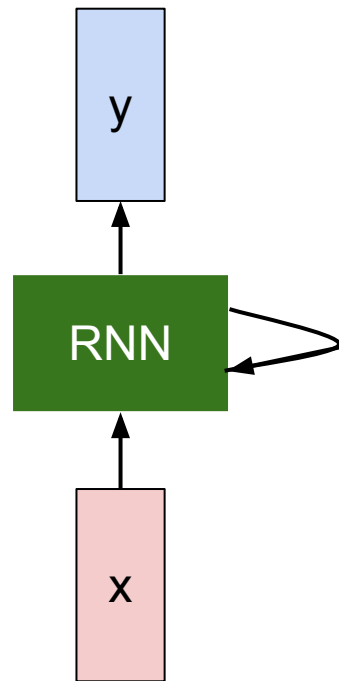
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state

some function with parameters W

old state

input vector at some time step



RNN output generation

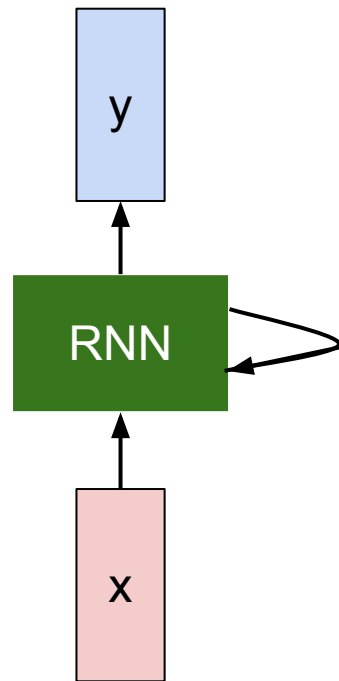
We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$\boxed{y_t} = \boxed{f_{W_{hy}}}(\boxed{h_t})$$

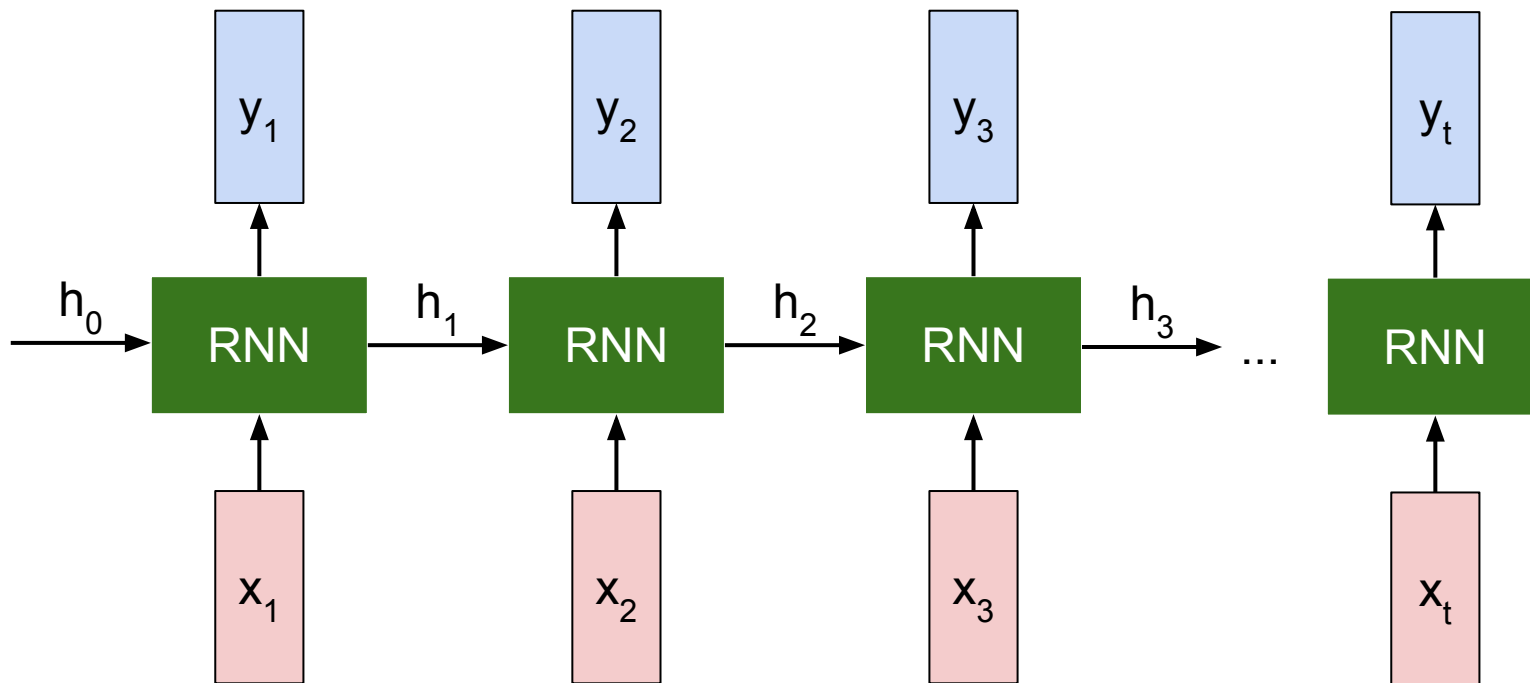
output

another function with parameters W_o

new state



Recurrent Neural Network

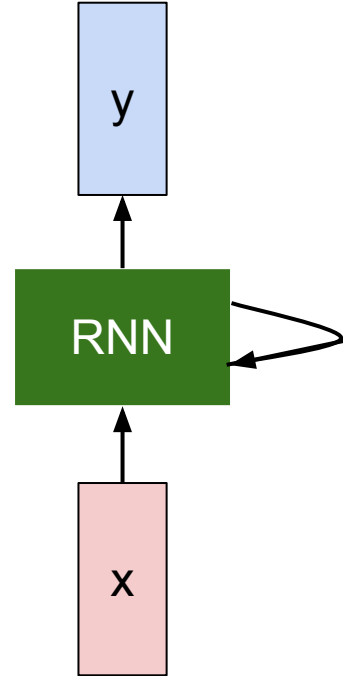


Recurrent Neural Network

We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

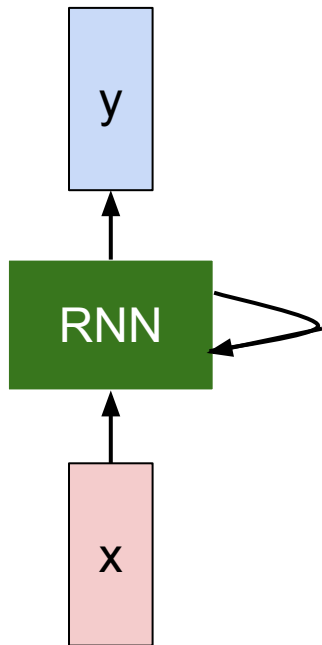
$$h_t = f_W(h_{t-1}, x_t)$$

Notice: the same function and the same set of parameters are used at every time step.



(Simple) Recurrent Neural Network

The state consists of a single “hidden” vector h :



$$h_t = f_W(h_{t-1}, x_t)$$

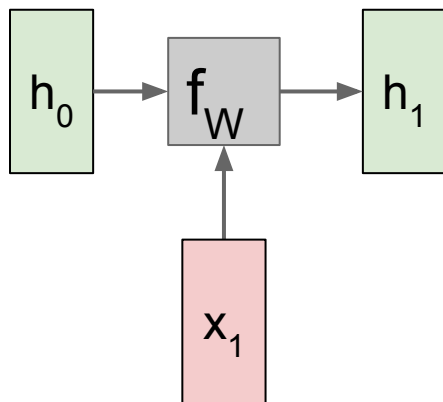


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

Sometimes called a “Vanilla RNN” or an “Elman RNN” after Prof. Jeffrey Elman

RNN: Computational Graph

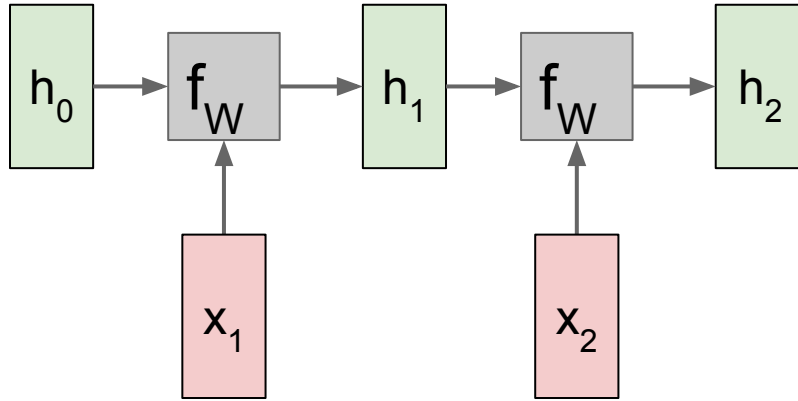


Tanh (гиперболический тангенс) — это функция активации, часто используемая в нейронных сетях, особенно в рекуррентных нейронных сетях (RNN). Она принимает значения на входе и "сжимает" их в диапазон от -1 до 1.

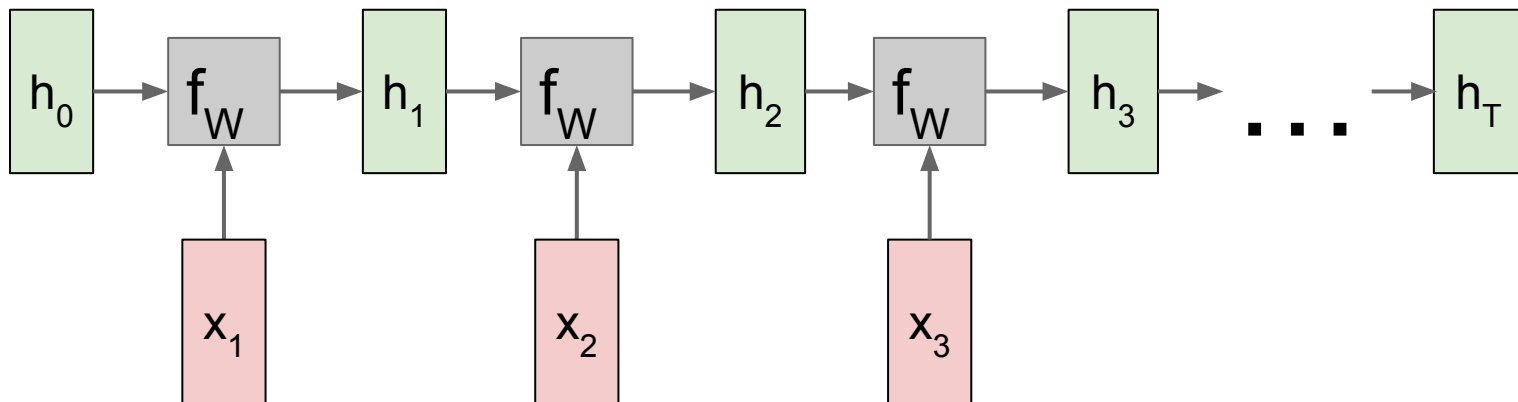
Wx — матрица весов для входа, связывает текущий вход x_t с текущим скрытым состоянием h_t .

Wh — матрица весов для скрытого состояния, связывает скрытое состояние на предыдущем временном шаге h_{t-1} с текущим скрытым состоянием h_t .

RNN: Computational Graph

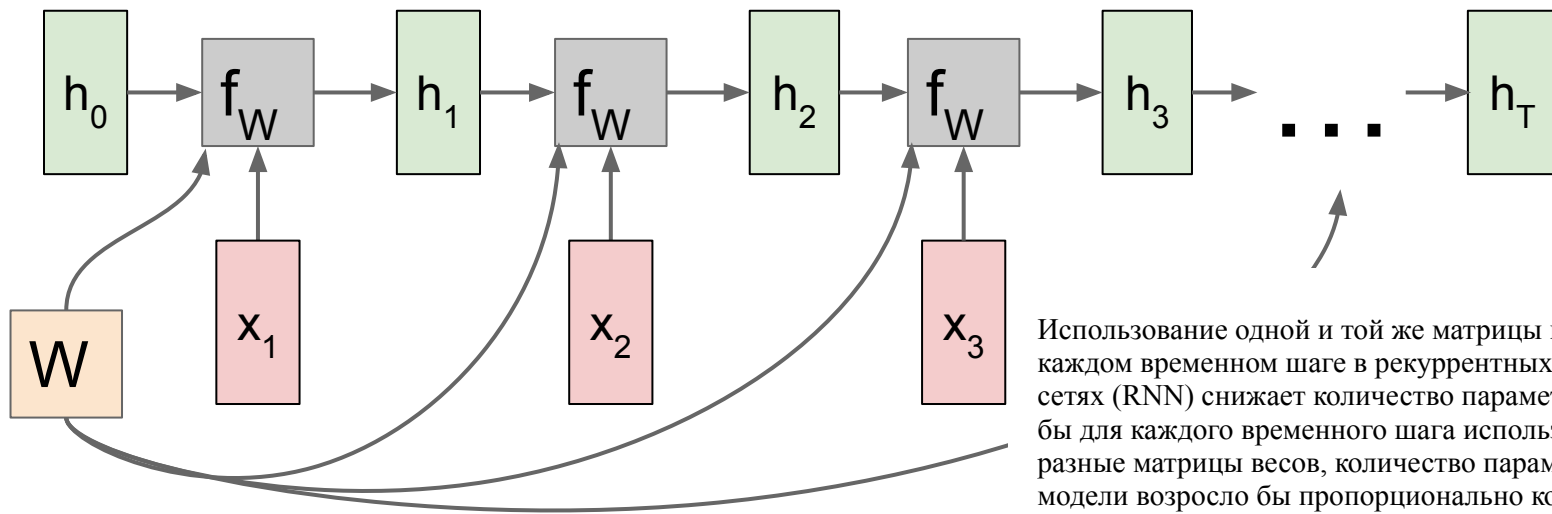


RNN: Computational Graph



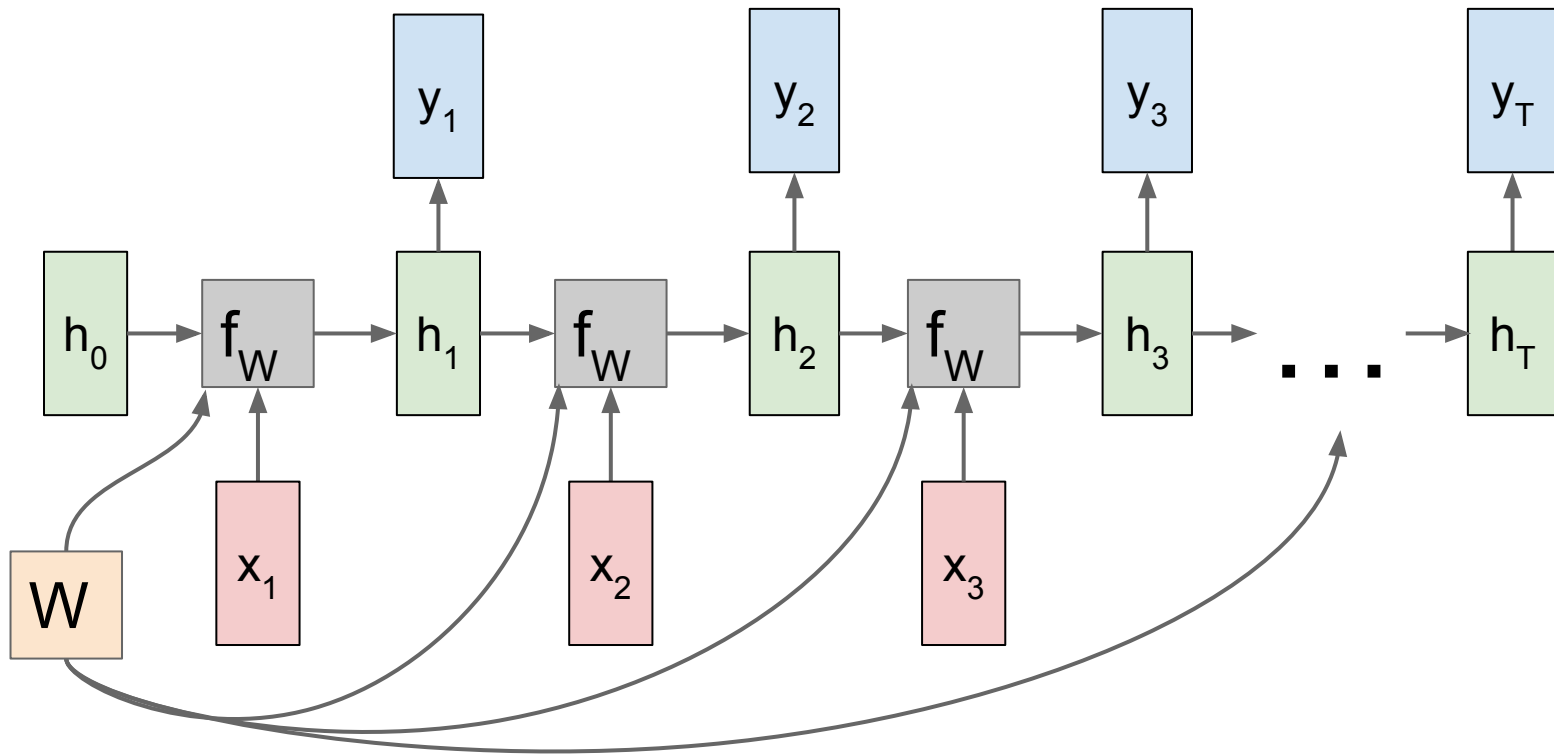
RNN: Computational Graph

Re-use the same weight matrix at every time-step

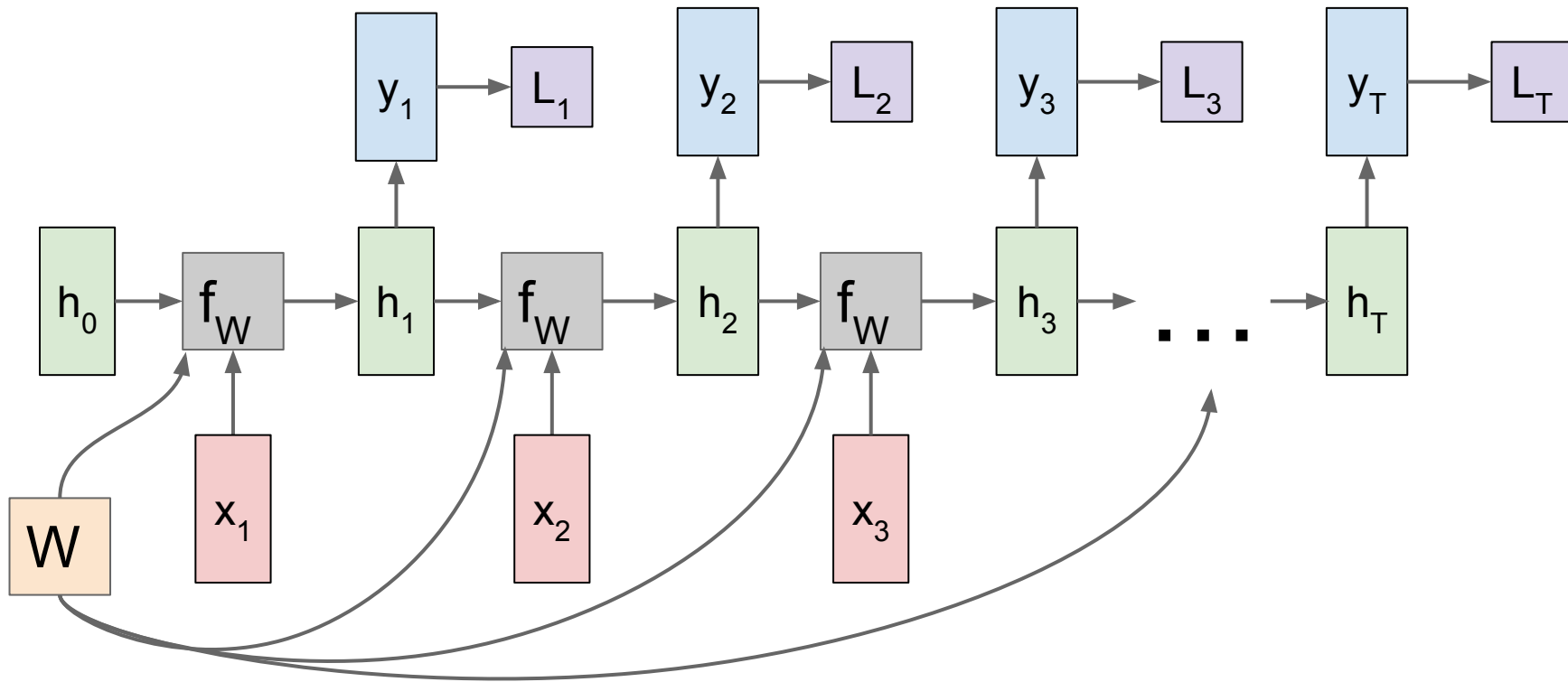


Использование одной и той же матрицы весов на каждом временном шаге в рекуррентных нейронных сетях (RNN) снижает количество параметров: Если бы для каждого временного шага использовались разные матрицы весов, количество параметров в модели возросло бы пропорционально количеству временных шагов. Это сделало бы обучение модели более ресурсоемким и сложным.

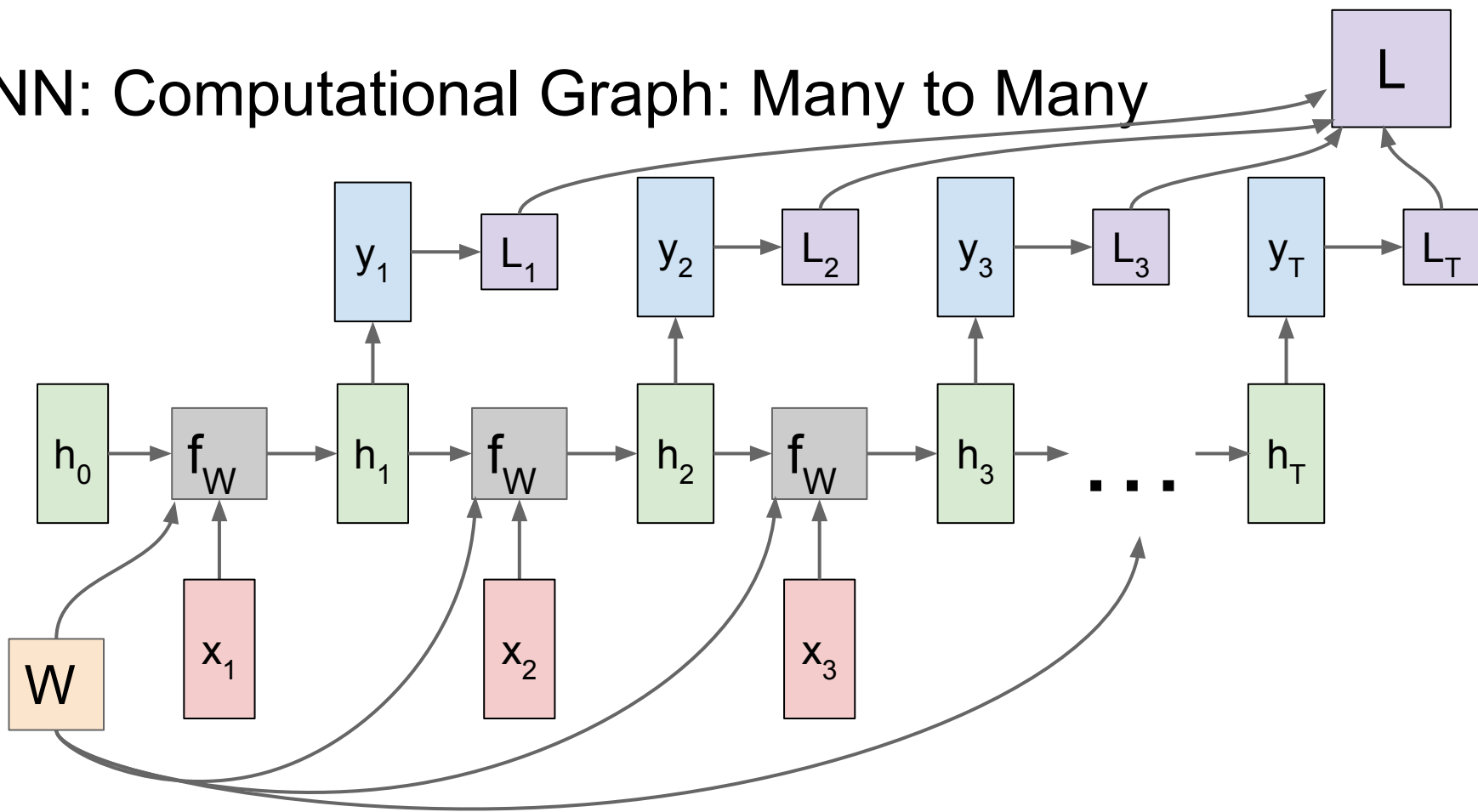
RNN: Computational Graph: Many to Many



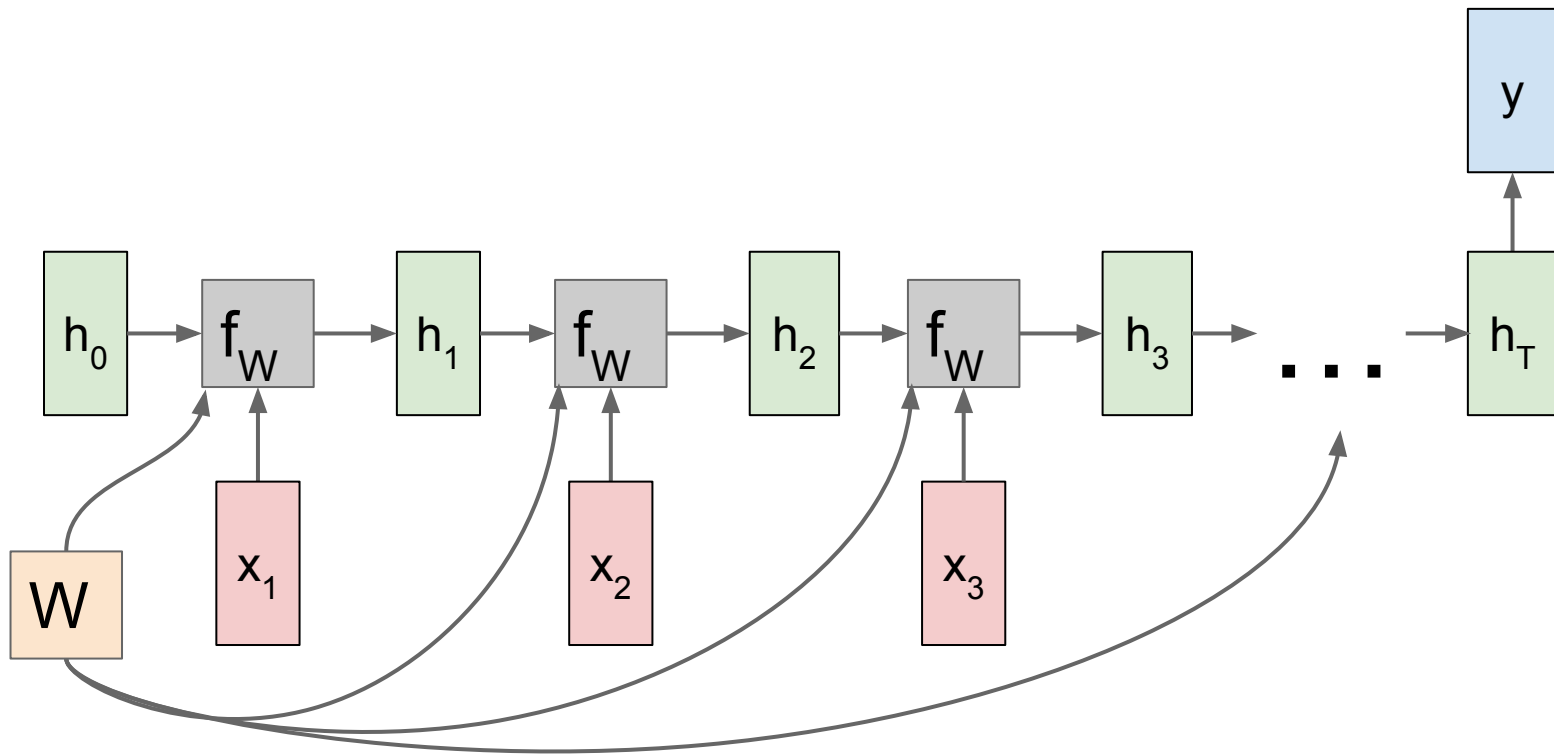
RNN: Computational Graph: Many to Many



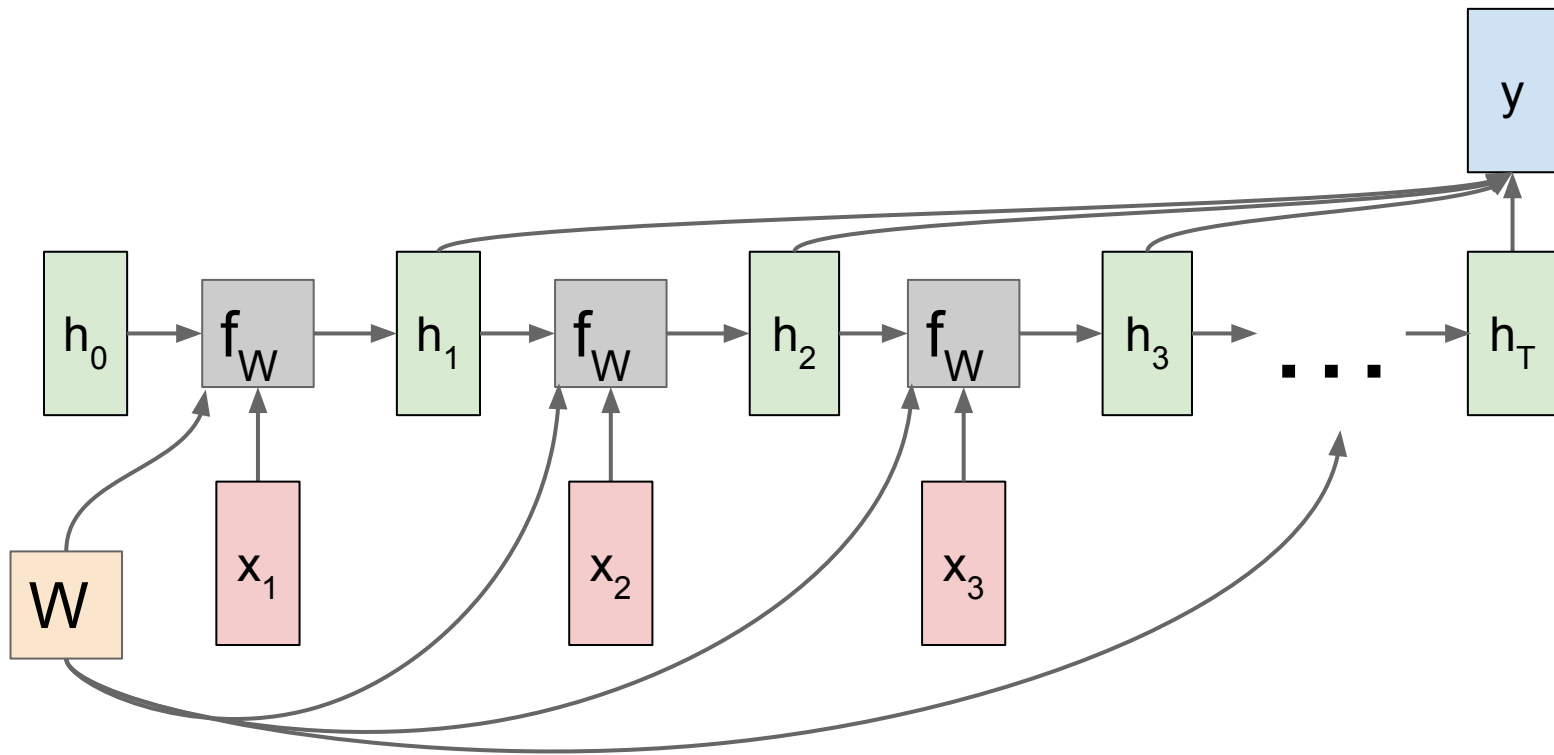
RNN: Computational Graph: Many to Many



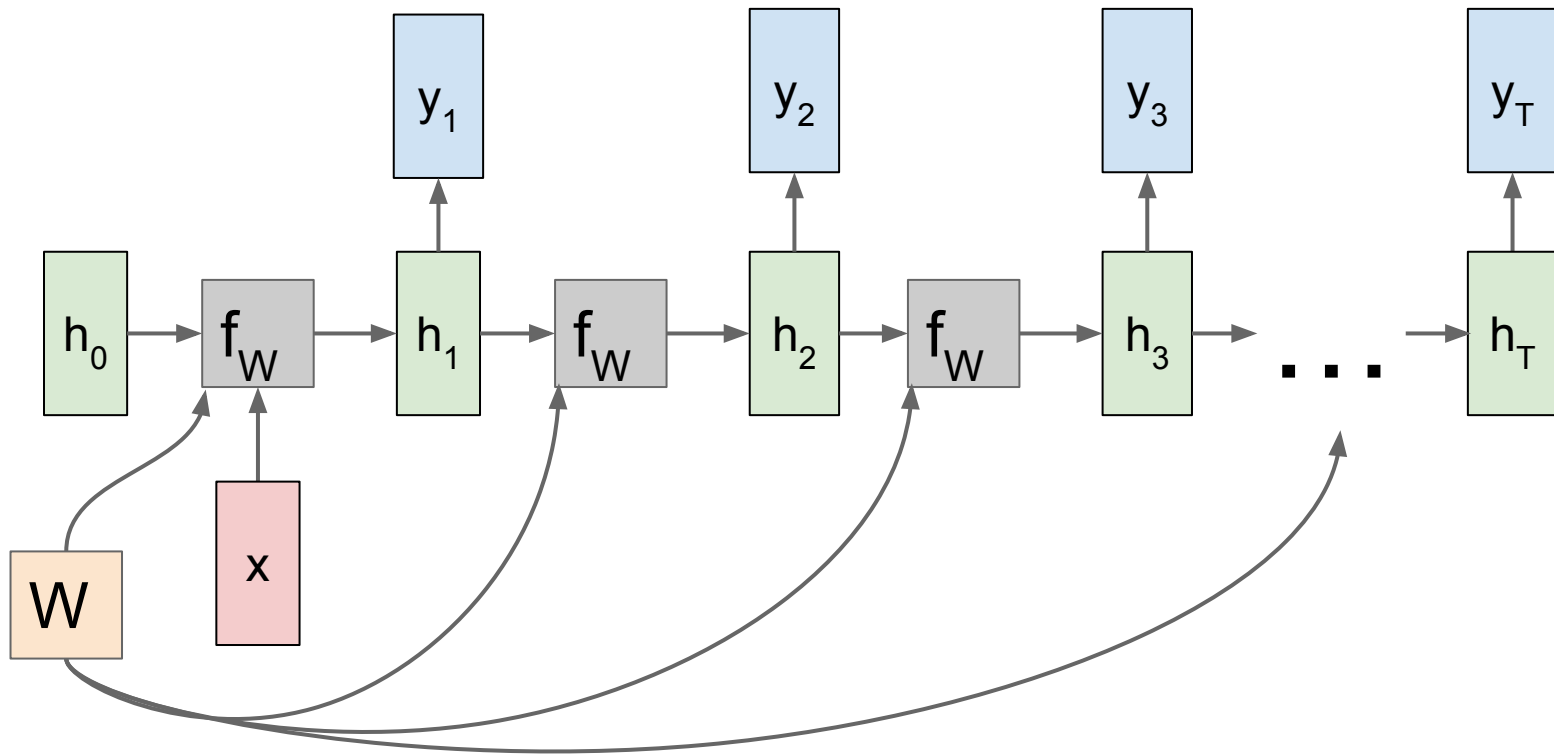
RNN: Computational Graph: Many to One



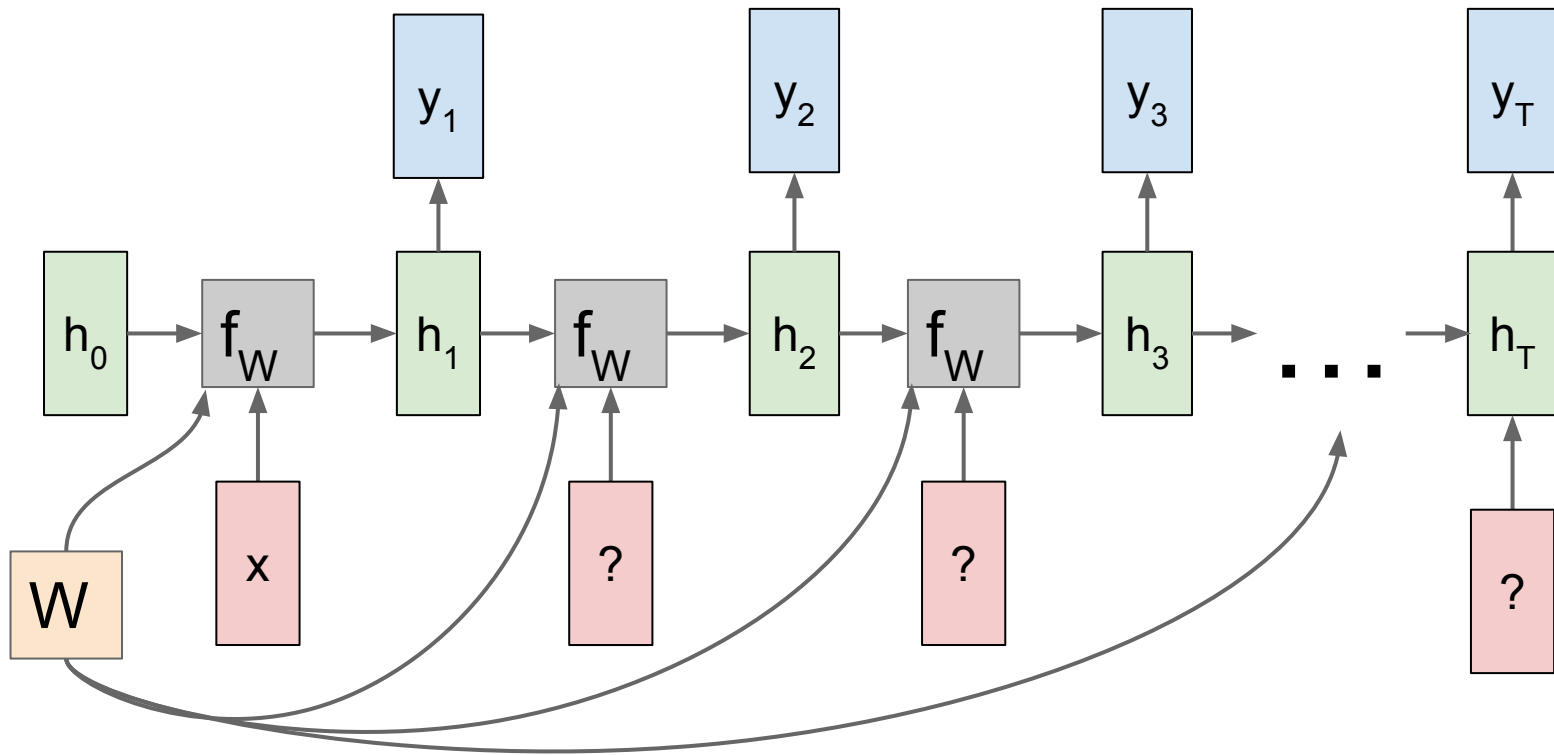
RNN: Computational Graph: Many to One



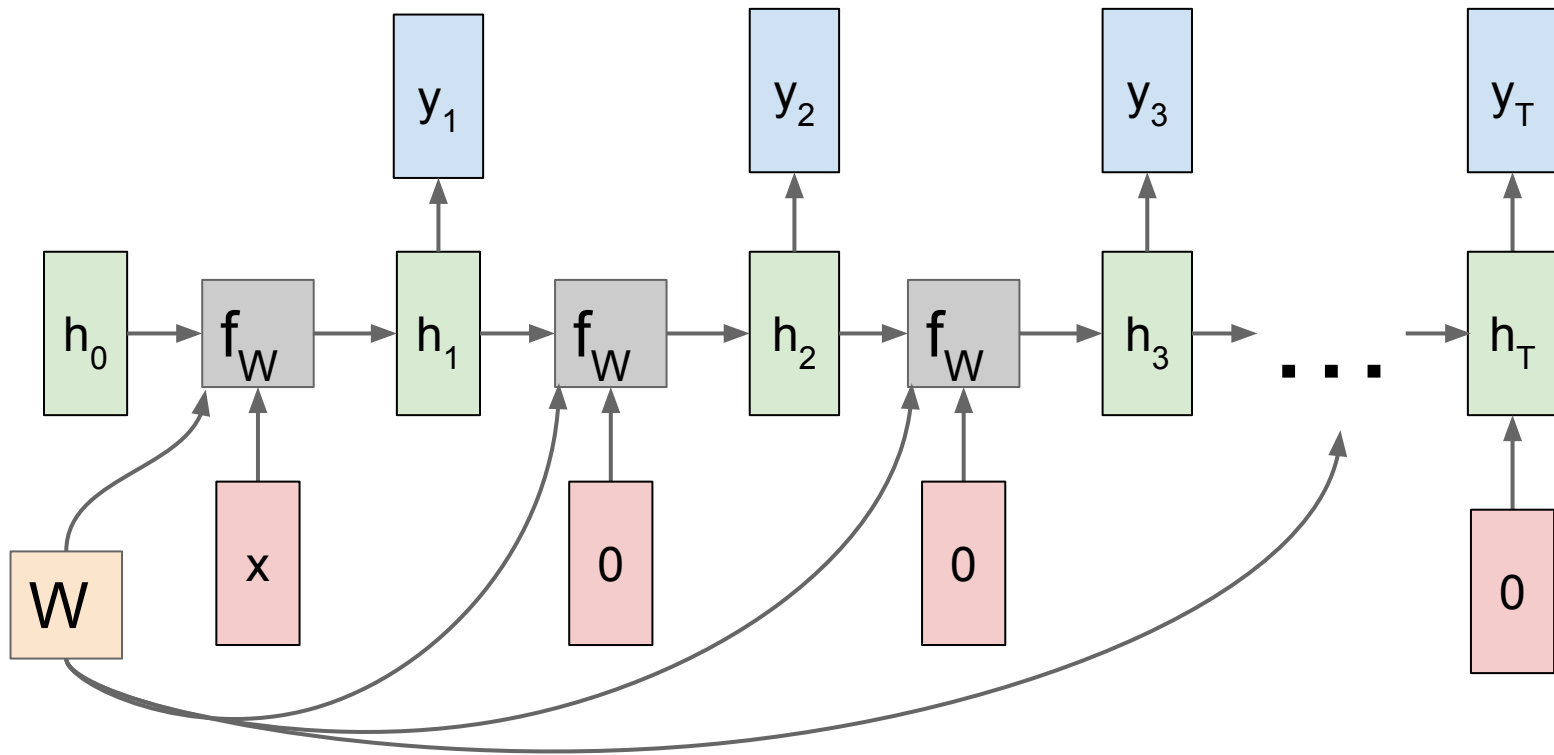
RNN: Computational Graph: One to Many



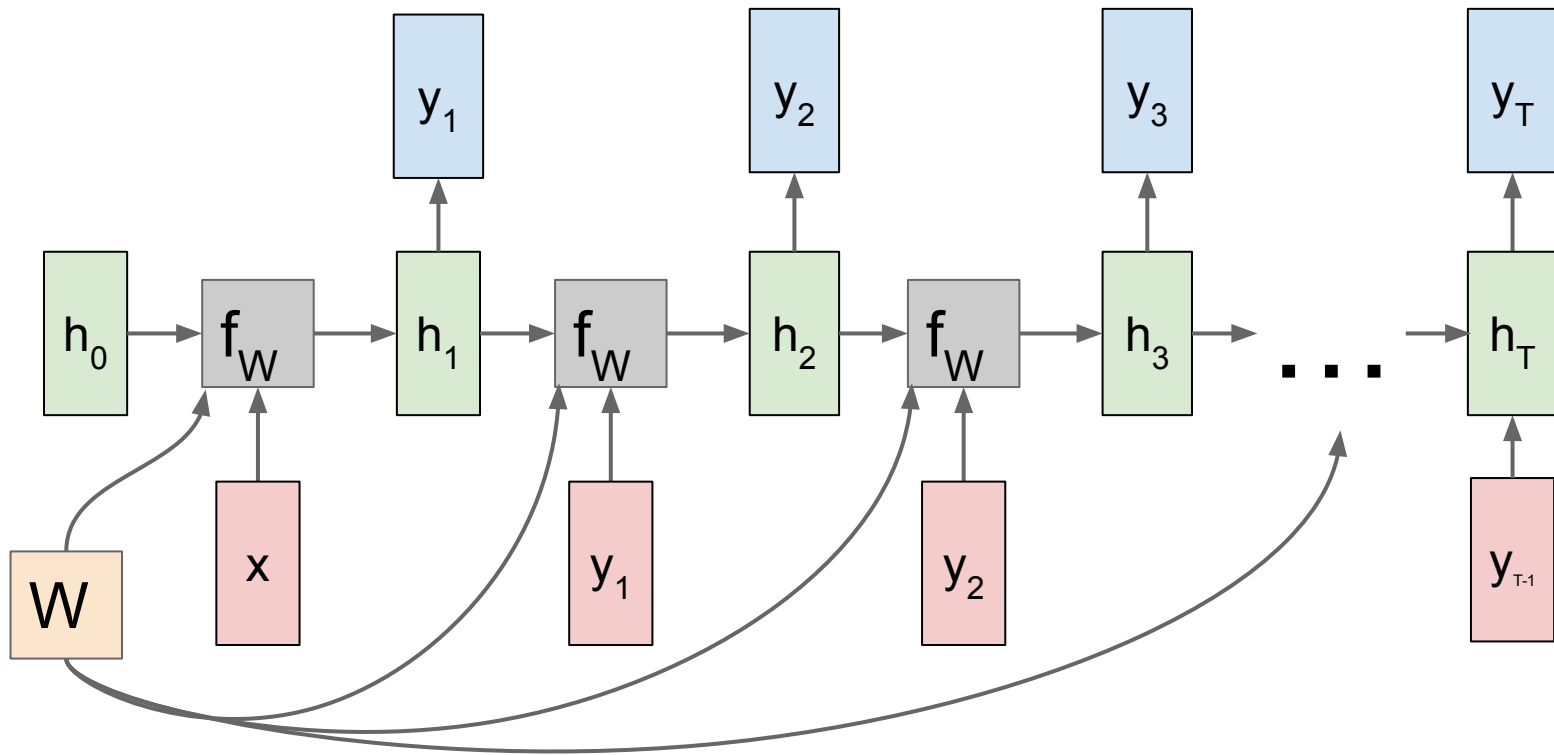
RNN: Computational Graph: One to Many



RNN: Computational Graph: One to Many



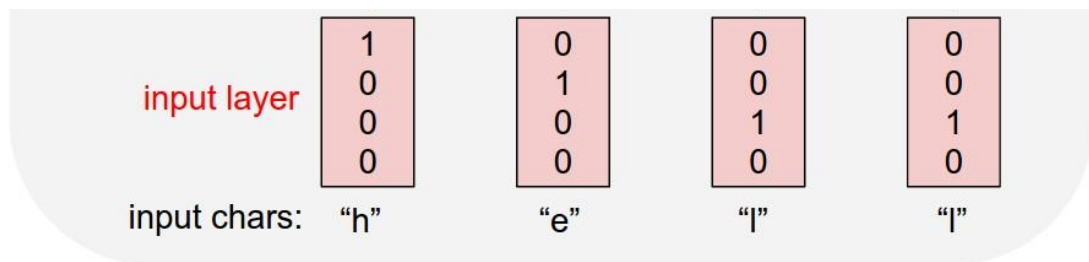
RNN: Computational Graph: One to Many



Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

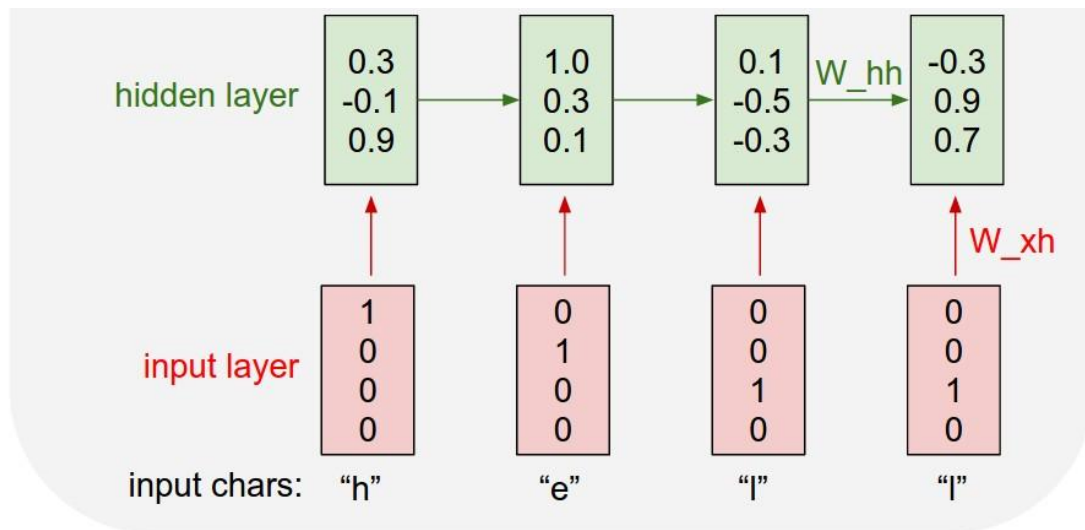


Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

Example training
sequence:
“hello”

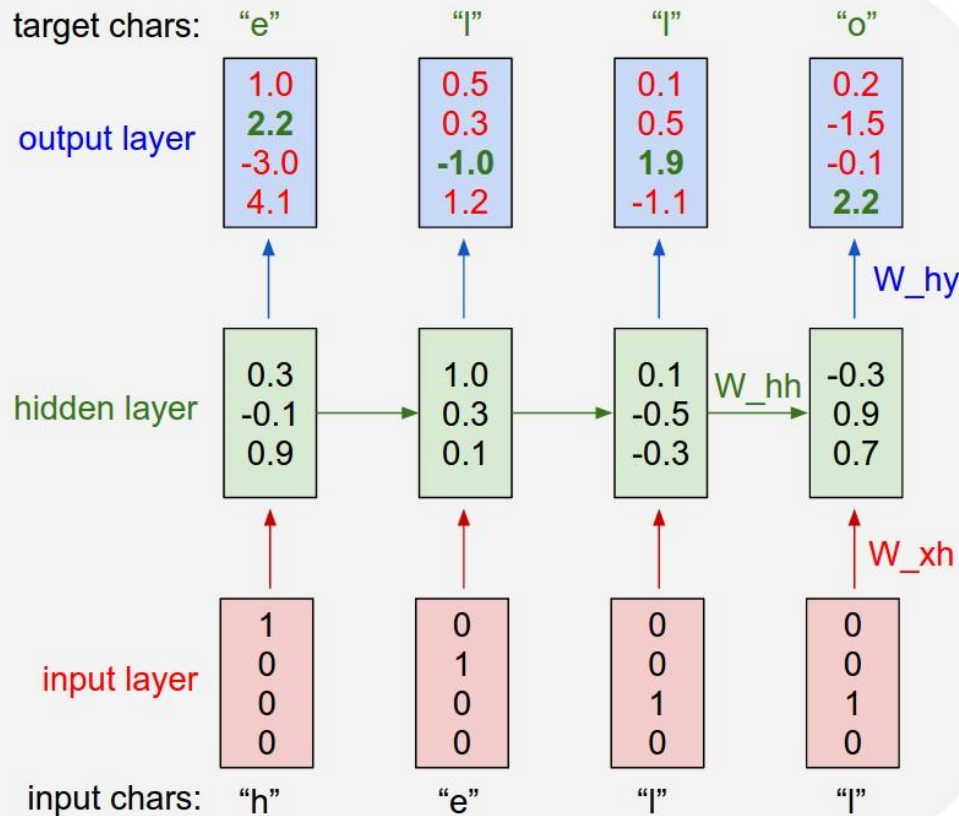
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



Example: Character-level Language Model

Vocabulary:
[h,e,l,o]

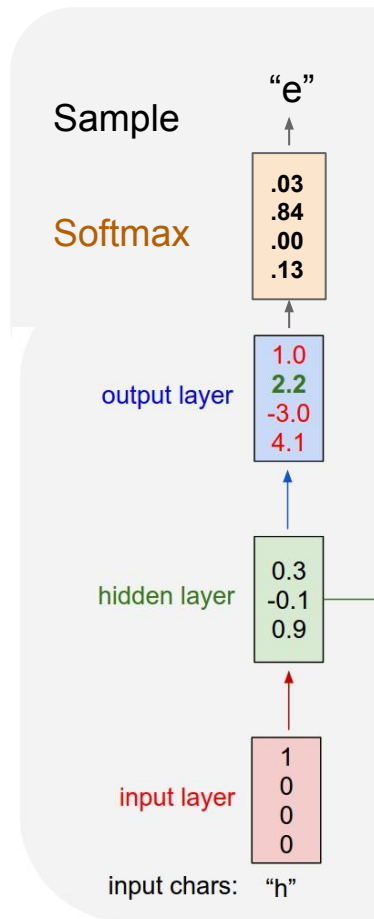
Example training
sequence:
“hello”



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

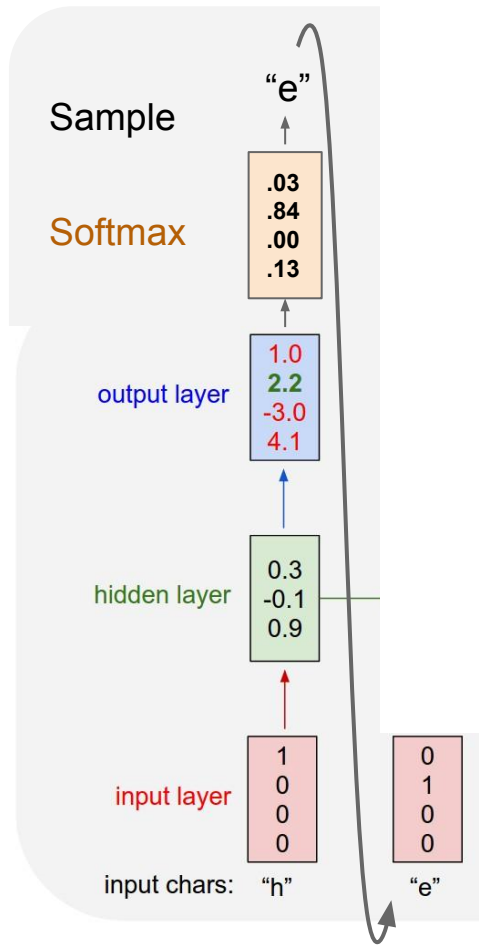
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

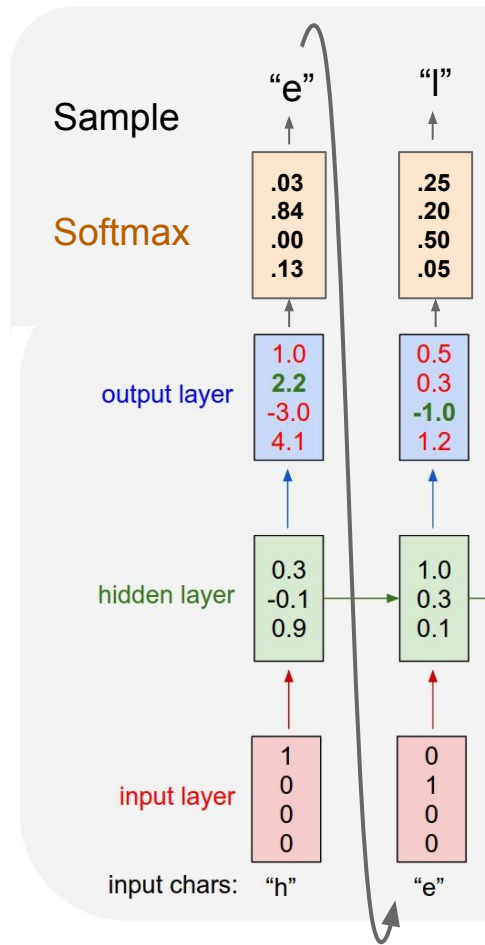
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

Vocabulary:
[h,e,l,o]

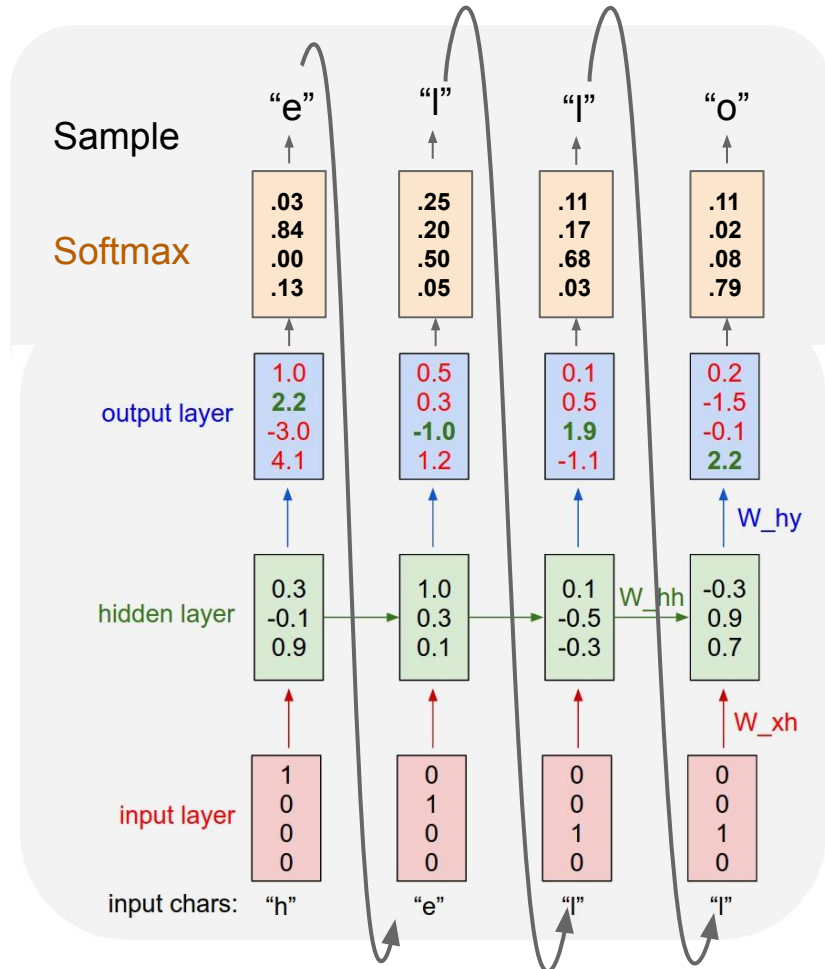
At test-time sample
characters one at a time,
feed back to model



Example: Character-level Language Model Sampling

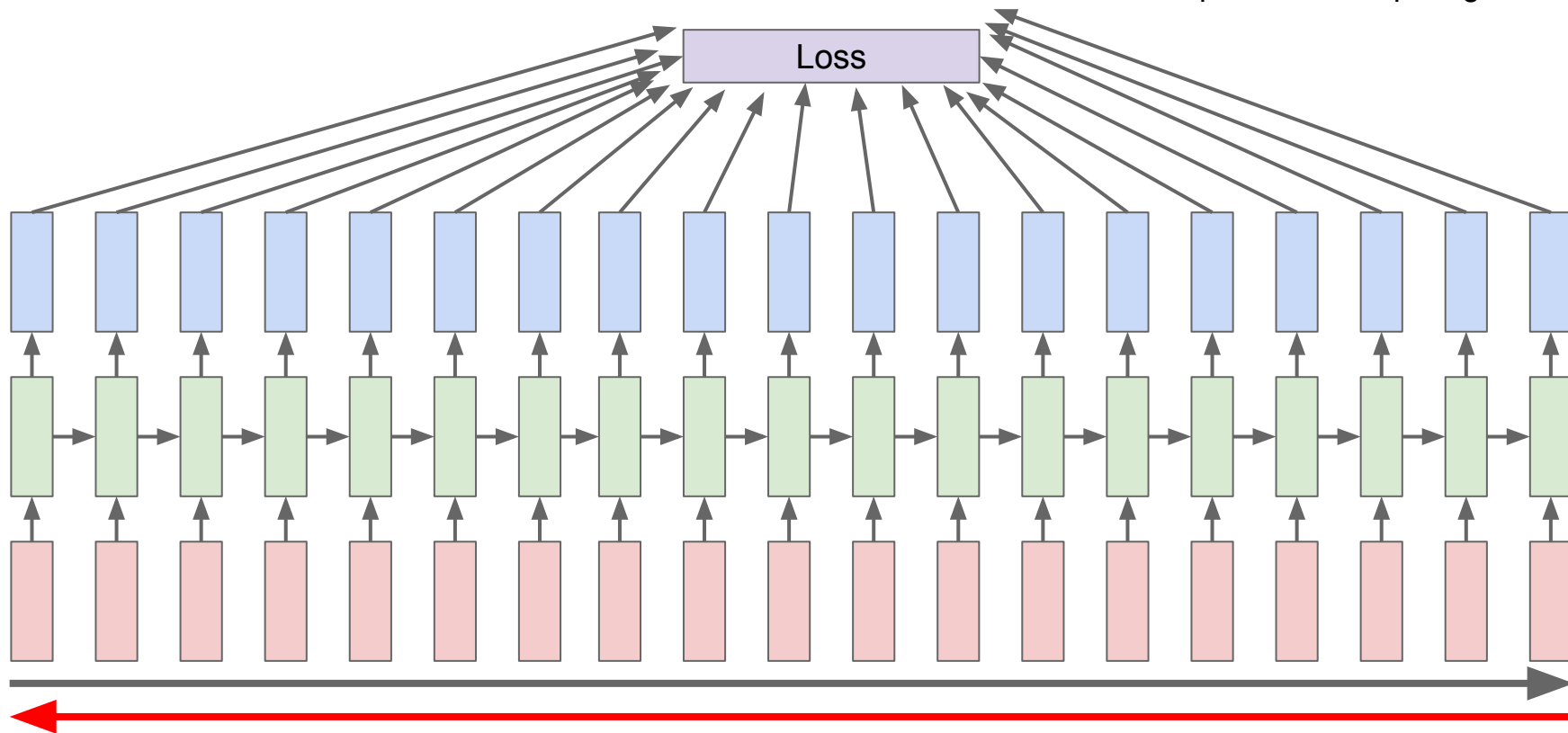
Vocabulary:
[h,e,l,o]

At test-time sample
characters one at a time,
feed back to model

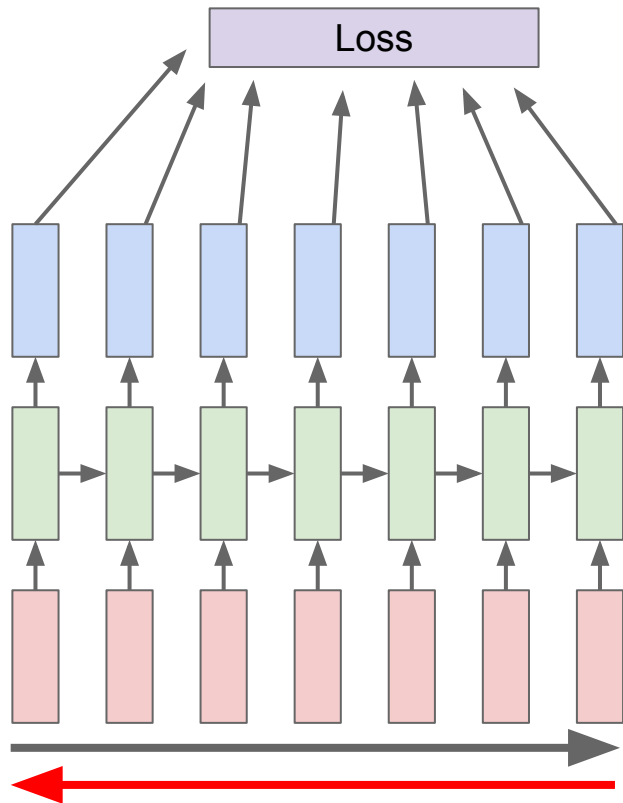


Backpropagation through time

Forward through entire sequence to compute loss, then backward through entire sequence to compute gradient

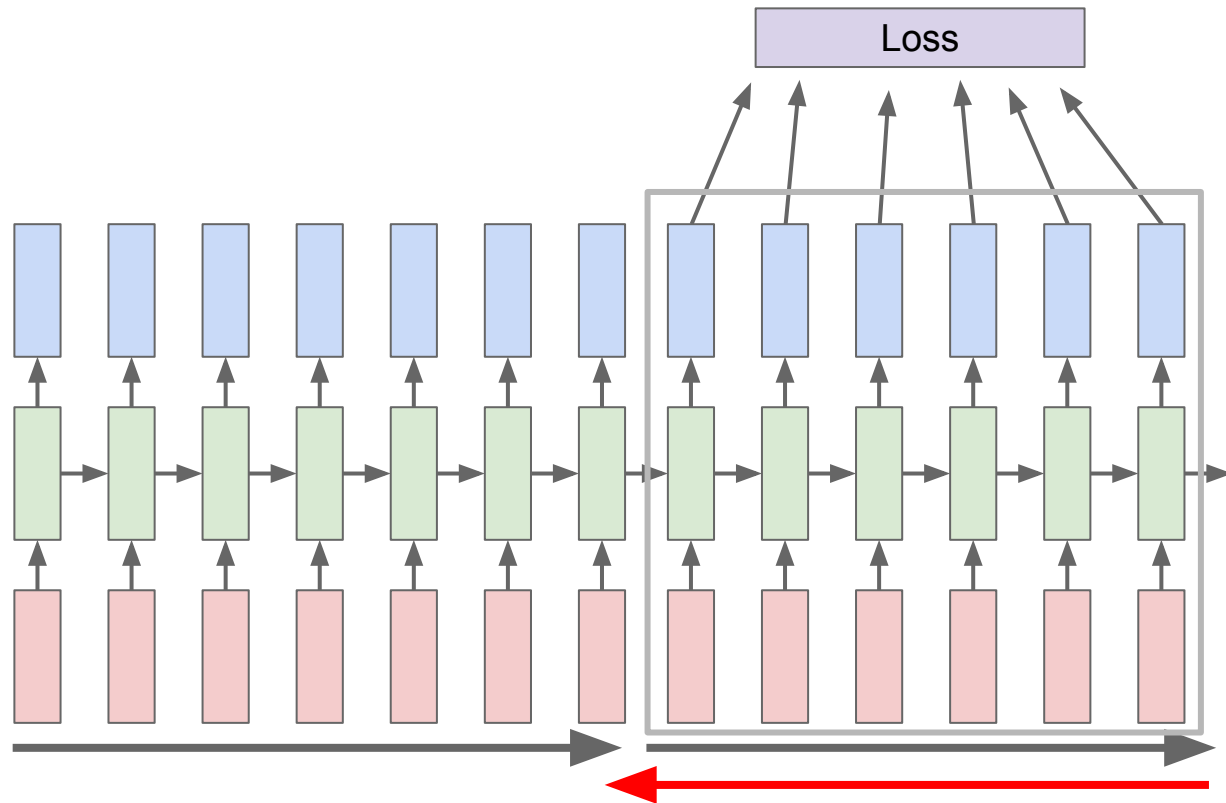


Truncated Backpropagation through time



Run forward and backward through chunks of the sequence instead of whole sequence

Truncated Backpropagation through time



Carry hidden states forward in time forever, but only backpropagate for some smaller number of steps

Truncated Backpropagation through time

